

The Agentic Router, extended: sub-query division and an RBAC-aware cache

A compound question now splits, routes each piece on its own, and comes back as one answer with separate citations. A semantic cache makes repeats faster and never crosses a role boundary.

Kurt Lozier · September 25, 2026 · Agentic_Router.ipynb, run end to end

3 of 3

check-yourself cases pass
(Part 1, required)

5 of 5

checks pass
(Part 1 cases, parser, Bonus self-check)

2944 → 1227 ms

bob's repeat question,
cache MISS then HIT

0 tokens

offline harness proves the logic
with a fake LLM and fake search

Both stubs are implemented, the notebook runs green, and the leak test passes.

REQUIRED · GRADED

Sub-query division

Split a compound question, route each sub-query separately, compose one answer that keeps every citation.

```
function agentic_rag_multi()
```

CHECK

1 question, 1 route, no compose call

2 sub-queries, both 10K_DOCUMENT_QUERY

2 sub-queries, different routes

Malformed, fenced, prose-wrapped or empty split degrades to one query

Bonus self-check: MISS, HIT, leak test, paraphrase leak, unknown user, shared source

BONUS · OPTIONAL

RBAC-aware semantic cache

Make repeat questions fast without ever serving one role's answer to another.

```
function secure_agentic_rag_cached()
```

RESULT

Run end to end, all checks green

Real OpenAI, Qdrant and SerpApi calls. Logic proved first offline at zero token cost.

cells above the stubs: **not modified**

SOURCE IN THE NOTEBOOK

RESULT

Part 1 check-yourself table

pass

Part 1 check-yourself table

pass

Part 1 check-yourself table

pass

Parser robustness cell

pass

run_self_check()

pass

One question, one route: an LLM router picks **one of three sources** and answers from it.

ROUTER	OPENAI_QUERY	10K_DOCUMENT_QUERY	INTERNET_QUERY
route_query()	→ OpenAI docs index	→ 10-K index	→ Live web search
LLM returns JSON: action, reason, answer.	Qdrant collection of OpenAI Agents documentation.	Qdrant collection of Lyft 2024 and Uber 2021 filings.	SerpApi, for anything current or general.

Stub 1 · Part 1, required

- Write **agentic_rag_multi(user_query)**.
- Split with **sub_queries()**; route each piece separately.
- Compose one answer; keep each sub-answer's citations.
- Single question: no regression, no extra LLM calls beyond the split.
- Parse the split's JSON defensively; fall back to one query.

Stub 2 · Bonus, optional

- Add a role-aware cache to **secure_agentic_rag()**.
- Pick partitioned or tagged; justify it.
- Fixed order: identity, route, permission, cache.
- Never cache a denial or a time-sensitive query.
- Pass the leak test.

Each sub-query runs the **same router and routes**; citations are renumbered so they never collide.

1 · SPLIT

sub_queries()

Defensive parser strips fences and prose.
Any failure falls back to the original query.
Cap of 5.

2 · ROUTE EACH

route_query()

→ Same router, same route table. Sub-queries may land on different sources.

3 · ANSWER EACH

Retrieve, then answer

→ Concurrent via `asyncio.gather`. Citations `[n]` become `[i.n]` for sub-query `i`.

4 · COMPOSE

One LLM call

→ Keep every `[i.n]` verbatim. A Sources footer maps each marker to its snippet.

Composed answer, case 2 (from the run)

"In 2021:

- Lyft revenue: \$3.208 billion (approximately \$3,208,323,000).[1.2]
- Uber revenue: \$17.455 billion (approximately \$17.5 billion).[2.1][2.2]"

Sources footer lists `[1.1]–[1.3]` and `[2.1]–[2.3]`, each with the 10-K snippet behind it

The single-question case

- One sub-query skips the compose call.
- LLM calls: split, route, answer.
- That is one more than **agentic_rag()**, the split the brief allows.
- Markers revert from `[1.n]` to `[n]`.

All three check-yourself cases pass, and the mixed case hits the 10-K index and live search.

QUERY	SUB-QUERIES	ROUTES CHOSEN	TIME	EXPECTED
what was uber revenue in 2021?	1	10K_DOCUMENT_QUERY	3 s	1 route, as agentic_rag()
what was lyft revenue in 2021 and what was uber revenue in 2021	2	10K_DOCUMENT_QUERY, 10K_DOCUMENT_QUERY	5 s	both 10-K
what was uber's 2021 revenue and what are the newest LLMs?	2	10K_DOCUMENT_QUERY, INTERNET_QUERY	7 s	different routes

Stretch: sequential vs concurrent

- Same mixed query, run both ways.
- Sequential 9.7 s, concurrent 7.1 s: **1.37× faster**.
- Blocking calls run in worker threads; Qdrant is already async.

Parser robustness, no API calls

- Fenced, prose-wrapped, malformed and empty splits tested.
- Every bad split degrades to one query instead of crashing.

Source: notebook output, check-yourself cells (times rounded to whole seconds; the notebook prints 2.8, 5.3 and 6.7 s) and the stretch cell

Permission is checked **before the cache is read**, so a denied user never reaches a cached answer.

1 · IDENTITY

Unknown user?

DENIED. No embedding, no cache, no LLM.

2 · ROUTE

→ `route_query()`

Which source does this question need?

3 · PERMISSION

→ `has_access()`

DENIED before any cache lookup. A denial is never cached.

4 · CACHE

→ **Same route, permitted rows**

HIT returns the stored answer. MISS runs the pipeline, then stores.

Design choice: tagged by source, not by role

- Each entry stores the route that produced it.
- A lookup sees only rows from routes the caller may read.
- alice and bob share OPENAI_QUERY rows; per-role caches would compute them twice.
- Role change: nothing to flush. Permissions are re-checked on every request.

What is never cached

- Denials.
- INTERNET_QUERY answers: live search is time-sensitive by nature.
- Time-sensitive wording: "latest", "now", "stock price" and similar.
- Anything the caller could not read (checked again on store).

bob's repeat drops from 2944 ms to 1227 ms; **alice is denied on the exact question and a paraphrase.**

#	USER	ROLE	QUERY	ROUTE	STATUS	SIM	MS
1	bob	finance_analyst	what was uber revenue in 2021?	10K_DOCUMENT_QUERY	MISS	–	2944
2	bob	finance_analyst	what was uber revenue in 2021?	10K_DOCUMENT_QUERY	HIT	1.0	1227
3	alice	engineer	what was uber revenue in 2021?	10K_DOCUMENT_QUERY	DENIED	–	1378
4	alice	engineer	how much revenue did Uber make in 2021?	10K_DOCUMENT_QUERY	DENIED	–	1041
5	carol	unknown	how do I build an agent with the OpenAI Agents SDK?	–	DENIED	–	0
6	alice	engineer	how do I build an agent with the OpenAI Agents SDK?	OPENAI_QUERY	MISS	–	9352
7	alice	engineer	how do I build an agent with the OpenAI Agents SDK?	OPENAI_QUERY	HIT	1.0	910

Rows 3 and 4 are the leak test. bob's answer sits in the cache, yet alice never reaches it: the permission check fires first. carol is rejected in 0 ms, before any embedding or LLM call. Result: "All checks passed — cache is fast and does not leak across roles."

A fake LLM and fake search proved the logic at zero token cost before the live run.

What is real

- The notebook's own cells, executed from the .ipynb.
- nomic embeddings and the local Qdrant index.
- Router parsing, route table and RBAC rules.
- The Part 1 and Bonus cells, self-check included.

WHAT THE HARNESS ASSERTS

Single case keeps [n] citations, no [1.1]

Same-route case composes with [1.1] and [2.1]

Mixed case uses INTERNET_QUERY and 10K_DOCUMENT_QUERY

Bonus self-check: MISS, HIT, DENIED pattern, leak test included

What is faked

- OpenAI client: splits on "and", routes by keyword, echoes citation markers.
- SerpApi: returns a fixed web result.
- Hugging Face runs in offline mode. No network, no keys.

WHY IT MATTERS

No regression for one question

Renumbering survives the compose step

Each sub-query is routed separately

The cache cannot leak, even before real calls

Source: offline_harness.py; README.md "Offline verification". Its MISS / HIT / DENIED pattern matched the live run, with timings in tens of ms.

This works for a notebook; production needs a shared, tuned, expiring cache.

LIMITATION	PRODUCTION CHANGE
In-memory cache (Python lists): lost on restart, one process only	Shared store, e.g. Redis or a vector DB, across workers
Threshold not tuned: cosine distance 0.2; live HITs were exact repeats	Sweep on a paraphrase set; measure false hits and misses
No TTL or eviction; freshness rests on a keyword regex	TTL per source; invalidate 10-K rows when the index is rebuilt
Router runs before the permission check: a denial still costs about 1 s	Pass the role's allowed sources into the router prompt
Gates whole sources, not chunks	Qdrant payload filters, so restricted chunks never enter context
Brute-force numpy search	faiss.IndexFlatIP beyond about 10k rows
Audit log in memory	Append-only store; that trail is what an auditor needs

Single runs. Each timing is one sample. The 1.37× concurrent speed-up and the 2944 → 1227 ms cache gain are indicative, not averages.

Source: Bonus cell design notes; notebook Section 6 "Where this is still weak"; leak-test timings. README.md lists no limitations.

RECAP

Split, route, compose. Check permission before the cache.

Sub-query division answers compound questions with a citation trail per source. The RBAC-aware cache speeds up repeats and never serves an answer across a role boundary.

3 of 3

check-yourself cases pass,
including mixed routes

1.37x

concurrent vs sequential
sub-queries, one run

2 of 2

leak attempts denied:
exact and paraphrase

0 tokens

offline harness run
before the live one

Kurt Lozier · Hamza's FDE Bootcamp, Cohort 2 · Thank you